PL-11

# Entrega de prácticas

Algoritmia 2025-2026

Pedro Menéndez Cerón
UO307004

# Índice

Práctica 0 ..... 2
Práctica 11 ..... 5
Práctica 12 ..... 10
Práctica 2 ..... 12
Práctica 3 ..... 15
Práctica 4 ..... 19
Práctica 5 ..... 20
Práctica 6 ..... 21
Práctica 7 ..... 22

# Práctica 0

## Factor 1 (PythonA1 ordenador de clase):

| Número de valores | Tiempo (ms) |
|---|---|
| 5000 | 388 |
| 10000 | 1496 |
| 20000 | 6000 |
| 40000 | 23327 |
| 80000 | 90591 |

## Factor 2 (PythonA1 ordenador de casa):

| Número de valores | Tiempo (ms) |
|---|---|
| 5000 | 528 |
| 10000 | 2281 |
| 20000 | 9505 |
| 40000 | 40910 |
| 80000 | 161048 |

Procesador: Intel(R) Core(TM) i7-10875H CPU @ 2.30GHz (2.30 GHz)

## Factor 3 (JavaA1):

| Número de valores | Tiempo con optimización (ms) | Tiempo sin optimización (ms) |
|---|---|---|
| 5000 | 38 | 107 |
| 10000 | 88 | 369 |
| 20000 | 340 | 1503 |
| 40000 | 1359 | 6326 |
| 80000 | 5531 | 27132 |

Con la ejecución del algoritmo A1 en java se puede observar como el tiempo se ha reducido en gran medida, tanto con la ejecución optimizada como sin optimizar, incluso comparándolo con el tiempo obtenido en el ordenador de clase que sacaba mejores resultados.

## Factor 4:

- PythonA2:

| Número de valores | Tiempo (ms) |
|---|---|
| 5000 | 76 |
| 10000 | 290 |
| 20000 | 1062 |
| 40000 | 3896 |
| 80000 | 14407 |

- PythonA3

| Número de valores | Tiempo (ms) |
|---|---|
| 5000 | 35 |
| 10000 | 134 |
| 20000 | 542 |
| 40000 | 1952 |
| 80000 | 7268 |

- PythonA4

| Número de valores | Tiempo (ms) |
|---|---|
| 5000 | 0 |
| 10000 | 1 |
| 20000 | 3 |
| 40000 | 5 |
| 80000 | 11 |

- JavaA2

| Número de valores | Tiempo con optimización (ms) | Tiempo sin optimización (ms) |
|---|---|---|
| 5000 | 19 | 22 |
| 10000 | 11 | 43 |
| 20000 | 40 | 153 |
| 40000 | 145 | 572 |
| 80000 | 607 | 2291 |

- JavaA3

| Número de valores | Tiempo con optimización (ms) | Tiempo sin optimización (ms) |
|---|---|---|
| 5000 | 17 | 21 |
| 10000 | 6 | 33 |
| 20000 | 21 | 115 |
| 40000 | 73 | 426 |
| 80000 | 271 | 1658 |

- JavaA4

| Número de valores | Tiempo con optimización (ms) | Tiempo sin optimización (ms) |
|---|---|---|
| 5000 | 12 | 11 |
| 10000 | 1 | 1 |
| 20000 | 1 | 1 |
| 40000 | 1 | 1 |

| | | |
|---|---|---|
| 80000 | 3 | 3 |

Realizadas las mediciones, se puede concluir que los algoritmos realizados en java son considerablemente más rápidos que los programados en Python. A parte de esto, es importante remarcar que, aunque en el algoritmo A4 hecho en java parezca más rápido la ejecución sin optimización que la optimizada, esto es porque en las tablas no se recogen los tiempos para un mayor número de valores, ya que a partir de n=320000 se ve una gran diferencia entre el optimizado, que tardó 4ms y el no optimizado, que fueron 14ms. Otra conclusión que es destacable de los datos tomados es que, en java, los algoritmos más rápidos tardan más en la primera ejecución del algoritmo (para n=5000) que en las siguientes, fenómeno que no ocurre en Python.

# Práctica 11

Ejercicio 1: 18.446.744.073.709.551.616 – 1770228546317 ms -> 5,8x10^8 años

Este cálculo es el resultado de ver el número de valores que puede almacenar un entero de tipo long (2^64) y restarle el número de milisegundos que pasaron desde que la cuenta se puso a 0

Tiempo ejecución Vector2 sin optimización

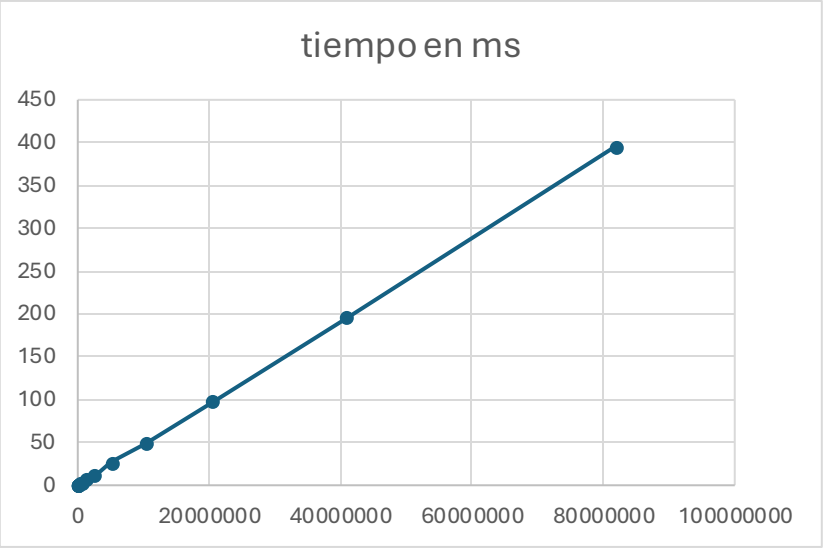
| Numero de valores | Tiempo ms |
|---|---|
| 1 millón | 5 |
| 10 millones | 48 |
| 50 millones | 241 |
| 100 millones | 505 |
| 200 millones | 959 |

En el ordenador de clase se empiezan a obtener tiempos fiables (>50ms) cuando el problema tiene un tamaño n>10000000 aproximadamente.

Tiempos de Vector3 sin optimización

```
n   tTiempo
10000   0
20000   0
40000   0
80000   1
160000  1
320000  2
640000  3
1280000 7
2560000 12
5120000 27
10240000        49
20480000        98
40960000        196
81920000        396

Fin de la medicion de tiempos *****
```




¿Por qué salen tiempos tan reducidos?

Salen tiempos tan cortos principalmente porque el algoritmo probado es bastante rápido y solo se está ejecutando una vez. Para conseguir tiempos fiables de este algoritmo habría que medir el tiempo que tarda en hacer varias repeticiones del mismo algoritmo, igual que se hace en el programa Vector4

Ejecutando la clase Vector4 con diferentes números de repeticiones se puede ver que por cada potencia de 10 que se añade a dicho número el tiempo también se multiplica aproximadamente por 10.

repeticiones = 100

| n | Tiempo |
|---|---|
| 10000 | 8 |
| 20000 | 13 |
| 40000 | 27 |
| 80000 | 53 |
| 160000 | 104 |
| 320000 | 207 |
| 640000 | 415 |

repeticiones = 1000

| n | Tiempo |
|---|---|
| 10000 | 67 |
| 20000 | 131 |
| 40000 | 261 |
| 80000 | 523 |
| 160000 | 1049 |
| 320000 | 2099 |
| 640000 | 4193 |

repeticiones = 10000

| n | Tiempo |
|---|---|
| 10000 | 653 |
| 20000 | 1317 |
| 40000 | 2607 |
| 80000 | 5251 |
| 160000 | 10465 |

repeticiones = 100000

| n | Tiempo |
|---|---|
| 10000 | 6393 |

repeticiones = 1000000

| n | Tiempo |
|---|---|
| 10000 | 70482 |

De esta forma, una ejecución del último ejemplo que aparece el guion (10 millones de repeticiones) del Vector4 tardaría unos 13068555270 milisegundos, que pasado a horas serían 3630 aproximadamente (este cálculo ha sido realizado midiendo el tiempo de esta ejecución para n=10000 y multiplicándolo por 2 para cada número de valores siguiente, que es el comportamiento que se ha demostrado de forma empírica que sigue esta clase aproximadamente):

| Numero de valores | Tiempo (ms) |
|---|---|

| | |
|---|---|
| 10000 | 797690 |
| 20000 | 1595380 |
| 40000 | 3190760 |
| 80000 | 6381520 |
| 160000 | 12763040 |
| 320000 | 25526080 |
| 640000 | 51052160 |
| 1280000 | 102104320 |
| 2560000 | 204208640 |
| 5210000 | 408417280 |
| 10240000 | 816834560 |
| 20480000 | 1633669120 |
| 40960000 | 3267338240 |
| 81920000 | 6534676480 |
| Tiempo total: | 13068555270 |

```
repeticiones = 10000000
n        Tiempo
10000    797690
```

Cuando el tamaño del problema se multiplica por 2, como ocurre en el Vector4 original, los tiempos se van duplicando, como se ha podido comprobar de manera práctica. Del mismo modo, si multiplicamos el tamaño del problema por 3 o por 4 los tiempos también se multiplicarán por 3 y por 4 respectivamente.

Tiempos para k=3 y 10 repeticiones

| Numero de valores | Tiempo (ms) |
|---|---|
| 10000 | 1 |
| 30000 | 2 |
| 90000 | 6 |
| 270000 | 18 |
| 810000 | 53 |
| 2430000 | 156 |
| 7290000 | 470 |
| 21870000 | 1485 |
| 65610000 | 4833 |

Tiempos para k=4 y 10 repeticiones

| Numero de valores | Tiempo (ms) |
|---|---|
| 10000 | 1 |
| 40000 | 4 |
| 160000 | 11 |
| 640000 | 42 |
| 2560000 | 171 |
| 10240000 | 720 |

| | |
|---|---|
| 40960000 | 2731 |

De todas formas, el tamaño máximo sigue capado en 81920000, por lo que el tiempo máximo sigue siendo para k=2 cuando n=81920000 donde el tiempo ronda los 5300 ms.

Estos tiempos son coherentes teniendo en cuenta la complejidad lineal del programa, pues al multiplicar n por una constante el tiempo tambien se multiplica por esa constante.

Los tiempos para los 4 métodos (suma, máximo, coincidencias1 y coincidencias2) ejecutados con diferentes n quedan reflejados en las siguientes tablas. Estos tiempos han sido medidos con 10 repeticiones para que estén más cerca de ser "tiempos fiables".

| n | Tsuma | Tmáximo |
|---|---|---|
| 10000 | 1 | 0 |
| 20000 | 1 | 1 |
| 40000 | 3 | 3 |
| 80000 | 5 | 6 |
| 160000 | 11 | 12 |
| 320000 | 22 | 25 |
| 640000 | 42 | 50 |
| 1280000 | 87 | 96 |
| 2500000 | 170 | 200 |
| 5120000 | 379 | 395 |
| 10240000 | 702 | 802 |
| 20480000 | 1396 | 2050 |
| 40960000 | 2743 | 3545 |
| 81920000 | 5374 | 8006 |

| n | Tcoincidencias1 | Tcoincidencias2 |
|---|---|---|
| 10000 | 6201 | 1 |
| 20000 | 26557 | 2 |
| 40000 | 108820 | 3 |
| 80000 | FdT | 6 |
| 160000 | FdT | 12 |
| 320000 | FdT | 24 |

| | | |
|---|---|---|
| 640000 | FdT | 51 |
| 1280000 | FdT | 109 |
| 2500000 | FdT | 217 |
| 5120000 | FdT | 403 |
| 10240000 | FdT | 833 |
| 20480000 | FdT | 1826 |
| 40960000 | FdT | 3675 |
| 81920000 | FdT | 7530 |

El ordenador en el que han sido tomados los tiempos tiene un procesador Intel(R) Core(TM) i7-10875H CPU @ 2.30GHz (2.30 GHz) y 32GB de RAM.

En conclusión, los tiempos obtenidos son coherentes, ya que los tres métodos con complejidad lineal tienen tiempos similares, y al recorrer prácticamente el mismo vector y hacer únicamente un bucle y pocas instrucciones básicas tiene sentido que sus tiempos sean muy reducidos; mientras que el método coincidencias1, que tiene una complejidad O(n^2), también es coherente que tenga tiempos muy superiores, al estar recorriendo vectores con un número elevado de elementos.

# Práctica 12

Ejercicio 1 (1 repetición)

| n | TBucle1 | TBucle2 | TBucle3 | TBucle4 |
|---|---|---|---|---|
| 100 | 0 | 0 | 1 | 1 |
| 200 | 0 | 0 | 4 | 6 |
| 400 | 0 | 3 | 16 | 42 |
| 800 | 0 | 13 | 67 | 324 |
| 1600 | 0 | 52 | 307 | 2794 |
| 3200 | 0 | 247 | 1253 | 20712 |
| 6400 | 1 | 928 | 4918 | 167044 |
| 12800 | 1 | 4182 | 19694 | Fdt |
| 25600 | 4 | 18936 | 83793 | Fdt |
| 51200 | 7 | 76221 | 348320 | Fdt |

Todos los tiempos concuerdan con las complejidades de los diferentes algoritmos, siendo O(nlogn) para Bucle1, O($n^2$logn) para Bucle2, O($n^2$logn) para Bucle3 y O($n^3$) para Bucle4.

Ejercicio 2 (1 repetición)

| n | TBucle5 | TBucle6 | TBucle7 |
|---|---|---|---|
| 100 | 1 | 7 | 20 |
| 200 | 4 | 65 | 284 |
| 400 | 22 | 570 | 4323 |
| 800 | 106 | 5020 | 66350 |
| 1600 | 514 | 43440 | Fdt |
| 3200 | 2430 | 374632 | Fdt |
| 6400 | 11346 | Fdt | Fdt |

Al igual que en el ejercicio anterior, todos los tiempos son coherentes teniendo en cuenta la complejidad de los algoritmos: O($n^2\log^2 n$) para Bucle5, O($n^3$logn) para Bucle6, O($n^4$) para Bucle7. Esto se puede comprobar porque al duplicar n, en cada caso aumenta el tiempo según la complejidad del algoritmo. Por ejemplo, en el Bucle7 con complejidad O($n^4$), el tiempo de n=1600 es aproximadamente 16 veces mayor que el tiempo para n=800, lo que cuadra con $2^4$.

Ejercicio 3 (para los cálculos se sustituyeron los t=0 por t=1)

| n | TBucle1 (t1) | TBucle2 (t2) | t1/t2 |
|---|---|---|---|
| 100 | 0 | 0 | 1 |
| 200 | 0 | 0 | 1 |
| 400 | 0 | 3 | 0,33333333 |
| 800 | 0 | 13 | 0,07692308 |
| 1600 | 0 | 52 | 0,01923077 |
| 3200 | 0 | 247 | 0,00404858 |

| | | | |
|---|---|---|---|
| 6400 | 1 | 928 | 0,00107759 |
| 12800 | 1 | 4182 | 0,00023912 |
| 25600 | 4 | 18936 | 0,00021124 |
| 51200 | 7 | 76221 | 9,1838E-05 |

El cociente de los tiempos sí concuerda con lo esperado, pues en el denominador se encuentra el algoritmo con mayor complejidad (Bucle2) y el cociente tiende a 0.

Ejercicio 4 (para los cálculos se sustituyeron los t=0 por t=1)

| n | TBucle3 | TBucle2 | t3/t2 |
|---|---|---|---|
| 100 | 1 | 0 | 1 |
| 200 | 4 | 0 | 4 |
| 400 | 16 | 3 | 5,33333333 |
| 800 | 67 | 13 | 5,15384615 |
| 1600 | 307 | 52 | 5,90384615 |
| 3200 | 1253 | 247 | 5,07287449 |
| 6400 | 4918 | 928 | 5,29956897 |
| 12800 | 19694 | 4182 | 4,70923003 |
| 25600 | 83793 | 18936 | 4,42506337 |
| 51200 | 348320 | 76221 | 4,5698692 |

Los resultados cuadran con los esperados, ya que el cociente de los tiempos no es exactamente constante pero ronda por un intervalo reducido de entre (4,5-6) aproximadamente.

Ejercicio 5

| n | Bucle4-Python | Bucle4-Java | Python/java |
|---|---|---|---|
| 200 | 35 | 8 | 4,375 |
| 400 | 263 | 63 | 4,17460317 |
| 800 | 2343 | 491 | 4,77189409 |
| 1600 | 19837 | 4179 | 4,74682939 |
| 3200 | 211048 | 35104 | 6,0120784 |
| 6400 | Fdt | Fdt | - |

Ocurre lo mismo que en el ejercicio anterior, los resultados oscilan en el intervalo (4-6) lo que demuestra que java es más rápido con estos algoritmos.

# Práctica 2

Ejercicio 1 Algoritmo burbuja (sin optimización):

| n | T ordenado | T inverso | T aleatorio |
|---|---|---|---|
| 10000 | 429 | 874 | 873 |
| 20000 | 1698 | 3447 | 3488 |
| 40000 | 6676 | 13873 | 13882 |
| 80000 | 26890 | 55580 | 55734 |
| 160000 | 107558 | 219873 | 221679 |
| 320000 | Fdt | Fdt | Fdt |
| 640000 | Fdt | Fdt | Fdt |
| 1280000 | Fdt | Fdt | Fdt |

El algoritmo burbuja tiene una complejidad de $O(n^2)$ tanto en su mejor caso (que ya esté ordenado), como en el peor (ordenado de forma descendente) y en el intermedio. De todas formas, el número de intercambios que se producen sí que varía, ya que en el mejor caso no se hace ninguno a diferencia de los otros 2 casos.

Ejercicio 2 Algoritmo selección

| n | T ordenado | T inverso | T aleatorio |
|---|---|---|---|
| 10000 | 402 | 412 | 397 |
| 20000 | 1562 | 1610 | 1597 |
| 40000 | 6345 | 6455 | 6277 |
| 80000 | 25136 | 25899 | 25050 |
| 160000 | 100594 | 100903 | 99927 |
| 320000 | Fdt | Fdt | Fdt |
| 640000 | Fdt | Fdt | Fdt |
| 1280000 | Fdt | Fdt | Fdt |

Al igual que en el algoritmo burbuja, el de selección también tiene una complejidad de $O(n^2)$ en los 3 casos. En este algoritmo el orden inicial de los elementos del vector no influye en el número de operaciones, por lo que no tendría un caso mejor o uno peor. Esto se puede observar claramente en los tiempos obtenidos, ya que en los tres casos son muy similares, y en todos se puede ver la complejidad cuadrática.

Ejercicio 3 Algoritmo inserción

| n | T ordenado | T inverso | T aleatorio |
|---|---|---|---|
| 10000 | 1 | 701 | 340 |
| 20000 | 1 | 2771 | 1356 |
| 40000 | 1 | 11054 | 5457 |
| 80000 | 2 | 43564 | 21755 |
| 160000 | 3 | 173736 | 86239 |
| 320000 | 6 | Fdt | Fdt |
| 640000 | 12 | Fdt | Fdt |

| | | | |
|---|---|---|---|
| 1280000 | 24 | Fdt | Fdt |

En este algoritmo el mejor caso pasa a tener una complejidad O(n), mientras que el peor y el intermedio siguen teniendo O($n^2$). Los tiempos tan reducidos obtenidos con el vector ordenado (el mejor caso) se deben a que no se hacen intercambios de elementos y cada elemento solo se compara una vez. En el caso del vector ordenado a la inversa, que es el peor caso, se puede apreciar el crecimiento cuadrático y, al compararlo con el caso intermedio (también con complejidad cuadrática) se confirma que esta es la situación menos favorable.

Ejercicio 4 Algoritmo rápido

| n | T ordenado | T inverso | T aleatorio |
|---|---|---|---|
| 10000 | 2 | 3 | 4 |
| 20000 | 5 | 6 | 8 |
| 40000 | 10 | 13 | 17 |
| 80000 | 23 | 26 | 34 |
| 160000 | 49 | 52 | 72 |
| 320000 | 101 | 110 | 156 |
| 640000 | 211 | 230 | 320 |
| 1280000 | 441 | 484 | 702 |

El Quicksort tiene complejidad O(n logn) en el mejor caso y en el intermedio, y O($n^2$) en el peor caso. Este peor caso se da cuando el vector ya está ordenado y el pivote es el primer elemento. En los casos medidos no se recoge ese peor caso por la forma de seleccionar el pivote, y por ello los resultados obtenidos son coherentes, pues se aprecia la complejidad cuasilineal en los 3 casos.

Para el algoritmo burbuja $t2 = \frac{81920000^2 * 55734}{80000^2}$ = 5,8441*$10^{10}$ ms, que pasado a dias serían 676,4.

Para el algoritmo selección $t2 = \frac{81920000^2 * 25050}{80000^2}$ = 2,6266*$10^{10}$ ms, que pasado a dias serían 304,01.

Para el algoritmo inserción $t2 = \frac{81920000^2 * 21755}{80000^2}$ = 2,2812*$10^{10}$ ms, que pasado a dias serían 264,03.

Para el algoritmo rápido $t2 = \frac{81920000 * \log(81920000) * 34}{80000 * \log(80000)}$ = 56191,62 ms, que pasado a dias serían 6,5*$10^{-4}$.

Ejercicio 5 Ejecutados con 30000 repeticiones

| n | T rápido | T inserción | T rápido/ t inserción |
|---|---|---|---|

| | | | |
|---|---|---|---|
| 10 | 35 | 11 | 3,1818 |
| 15 | 56 | 27 | 2,0741 |
| 20 | 81 | 48 | 1,6875 |
| 25 | 121 | 75 | 1,6133 |
| 30 | 137 | 110 | 1,2455 |
| 35 | 168 | 155 | 1,0839 |
| 40 | 195 | 183 | 1,0656 |
| 45 | 215 | 247 | 0,8704 |
| 50 | 267 | 280 | 0,9536 |
| 55 | 293 | 365 | 0,8027 |
| 60 | 306 | 411 | 0,7445 |
| 65 | 337 | 473 | 0,7125 |
| 70 | 378 | 555 | 0,6811 |
| 75 | 414 | 624 | 0,6635 |
| 80 | 448 | 696 | 0,6437 |
| 85 | 474 | 789 | 0,6008 |
| 90 | 519 | 881 | 0,5891 |
| 95 | 582 | 997 | 0,5838 |
| 100 | 557 | 1082 | 0,5148 |

El algoritmo rápido es peor que el de inserción hasta n = 45 aproximadamente, que es el primer tamaño para el que el algoritmo de inserción tarda más que el rápido.

Ejercicio opcional

| n | T combinado | T rápido | T combinado/T rápido |
|---|---|---|---|
| 10 | 22 | 35 | 0,6286 |
| 15 | 29 | 56 | 0,5179 |
| 20 | 53 | 81 | 0,6543 |
| 25 | 83 | 121 | 0,6860 |
| 30 | 112 | 137 | 0,8175 |
| 35 | 142 | 168 | 0,8452 |
| 40 | 200 | 195 | 1,0256 |
| 45 | 210 | 215 | 0,9767 |
| 50 | 251 | 267 | 0,9401 |
| 55 | 291 | 293 | 0,9932 |
| 60 | 321 | 306 | 1,0490 |
| 65 | 347 | 337 | 1,0297 |
| 70 | 398 | 378 | 1,0529 |
| 75 | 402 | 414 | 0,9710 |
| 80 | 454 | 448 | 1,0134 |
| 85 | 492 | 474 | 1,0380 |
| 90 | 499 | 519 | 0,9615 |
| 95 | 532 | 582 | 0,9141 |
| 100 | 574 | 557 | 1,0305 |

Viendo la comparación del algoritmo combinado con el rápido se puede observar como los primeros 35 valores aproximadamente son considerablemente más rápidos que con el algoritmo rápido, mientras que con valores superiores el algoritmo combinado ya ronda una velocidad similar al rápido.

# Práctica 3

Sustraccion1 tiene complejidad O(1). Deja de dar valores para n>8000

Sustraccion2 tiene complejidad $O(n^2)$. Deja de dar valores para n>8000

Estas clases dejan de dar valores porque se desborda la pila al realizar un número de llamadas a métodos muy grande.

Sustraccion3 tiene complejidad O(n)

Para n = 80, la clase Sustraccion3 tardaría aproximadamente 48422703193487572992 milisegundos ($2^{80-20}$ * 42, ya que 42ms era el tiempo para n=20), que pasado a años serían 1535473845,557 años. (1500 millones de años aproximadamente)

Tiempos de Sustraccion4

| n | Tiempos (ms) |
|---|---|
| 100 | 2 |
| 200 | 20 |
| 400 | 142 |
| 800 | 1142 |
| 1600 | 9191 |
| 3200 | 73252 |
| 6400 | Fdt |

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++){
            for(int j = 0; j < n; j++){
                cont++; // O(n^3)
            }
        } ;
        rec4 (n-1);
    }
}
```

Tiempos de Sustraccion5

| n | Tiempo (ms) |
|---|---|
| 30 | 396 |
| 32 | 1189 |
| 34 | 3592 |
| 36 | 10820 |
| 38 | 32280 |
| 40 | 97024 |
| 42 | Fdt |

```
public static void rec5 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        cont++;
        rec5 (n-2);
        rec5 (n-2);
        rec5 (n-2);
    }
}
```

Para n = 80, la clase Sustraccion5 tardaría aproximadamente 336000000000000 milisegundos ($3^{(80-30)/2}$ * 397, ya que 397ms era el tiempo para n=30), que pasado a años serían 10654 años.

Division1 tiene complejidad O(n)

Division2 tiene complejidad O(nlogn)

Division3 tiene complejidad $O(n^{\log_2 2}) = O(n)$

Tiempos de Division4

| n | Tiempo (ms) |
|---|---|
| 1000 | 9 |
| 2000 | 36 |
| 4000 | 144 |
| 8000 | 550 |
| 16000 | 2210 |
| 32000 | 9009 |
| 64000 | 35954 |
| 128000 | Fdt |

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=1;i<n;i++){
            for(int j = 1; j < n; j++){
                cont++ ;
            }
        }
        rec4 (n/2);
    }

    }
```

Tiempos de Division5

| n | Tiempo (ms) |
|---|---|
| 1000 | 26 |
| 2000 | 101 |
| 4000 | 402 |
| 8000 | 1623 |
| 16000 | 6534 |
| 32000 | 26281 |
| 64000 | 104176 |
| 128000 | Fdt |

Tiempos de la suma (medido en nanosegundos)

| n | TSuma1 | TSuma2 | TSuma3 |
|---|---|---|---|
| 100 | 473 | 1614 | 2917 |

Los 3 métodos tienen complejidad O(n). Se puede ver como el método implementado de forma recursiva con división es el más lento de los 3, siendo el más eficiente el método implementado de forma trivial.

Tiempos de Fibonacci

| n | TFibonacci1 | TFibonacci2 | TFibonacci3 | TFibonacci4 |
|---|---|---|---|---|
| 40 | 235ns | 492ns | 642ns | 4193ms |

Los 3 primeros métodos que calculan Fibonacci tienen una complejidad O(n), mientras que el último tiene complejidad $O(1,6^n)$. Estas complejidades se ven reflejadas en los tiempos obtenidos, ya que los 3 primeros métodos tienen tiempos similares, siendo el primero el más eficiente, mientras que el último es el peor de todos por la complejidad exponencial que representa.

Tiempos PuntosTrivial (optimizador activado)

| N | Tiempo (ms) |
|---|---|
| 128 | 1 |
| 256 | 1 |
| 512 | 2 |
| 1024 | 2 |
| 2048 | 4 |
| 4096 | 19 |
| 8192 | 60 |
| 16384 | 240 |
| 32768 | 989 |
| 65536 | 4054 |
| 131072 | 17771 |
| 262144 | 69511 |
| 524288 | Fdt |

Los tiempos si se sintonizan con la complejidad del algoritmo ($O(n^2)$), ya que cuando se duplica el tamaño del problema se cuadruplica el tiempo de ejecución

Tiempos PuntosDyV (optimizador activado)

| n | Tiempo (ms) |
|---|---|
| 8192 | 2 |
| 16384 | 4 |
| 32768 | 12 |
| 65536 | 28 |
| 131072 | 109 |
| 262144 | 97 |

| | |
|---|---|
| 524288 | 176 |
| 1048576 | 344 |
| 2097152 | 851 |
| 4194304 | 1910 |
| 8388608 | 5006 |
| 16777216 | 10932 |
| 33554432 | 26088 |
| 67108864 | 53940 |

La complejidad es O(nlogn), que se aproxima al comportamiento de los tiempos obtenidos, por lo que podemos ver un resultado coherente con el algoritmo implementado.

# Práctica 4

```java
public class ColoreoGrafo {

    static private String[] colores = {"red", "blue", "green", "yellow","orange", "purple","cyan", "magenta", "lime"};

    static public Map<String,String> realizarVoraz(Map<String,List<Long>> grafo){
        Map<String,String> coloreado = new HashMap<>();

        //Recorro todo el grafo
        for(String nodo : grafo.keySet()){

            List<String> coloresVecinos = new ArrayList<String>();

            //Recorro los vecinos del grafo seleccionado en esta iteracion
            for(Long vecino : grafo.get(nodo)){

                //Si el vecino ya estaba coloreado se guarda el color para no repetirlo
                if(coloreado.containsKey(vecino.toString())){
                    coloresVecinos.add(coloreado.get(vecino.toString()));
                }
            }

            //Se selecciona el primer color de la lista que no coincida con los de los vecinos
            String color = null;
            for(String c : colores){
                if(!coloresVecinos.contains(c)){
                    color = c;
                    break;
                }
            }

            //Guardo el nodo con su color
            coloreado.put(nodo,color);
        }

        return coloreado;
    }
}
```

El algoritmo, por lo general, tendría una complejidad de O(n), ya que a pesar de que hay dos bucles anidados en 2 puntos del programa el único bucle que recorre todos los nodos del grafo es el externo, y los otros dos no deberían hacer un número elevado de iteraciones, pues el segundo solo recorre los 9 colores disponibles como máximo y el primero el número de vecinos de cada nodo. Por lo general el número de vecinos por nodo es muy inferior al número de nodos, pero si

se diese el caso de que en un grafo el número de vecinos sea similar al de nodos llegaría a una complejidad $O(n^2)$ .

Tabla de tiempos obtenidos ejecutándolo con optimización

| N | T Coloreado |
|---|---|
| 8 | 38 |
| 16 | 2 |
| 32 | 2 |
| 64 | 4 |
| 128 | 3 |
| 256 | 5 |
| 512 | 9 |
| 1024 | 14 |
| 2048 | 30 |
| 4096 | 57 |
| 8192 | 108 |
| 16384 | 257 |
| 32768 | 941 |
| 65536 | 3279 |

A diferencia del resto, los dos últimos valores obtenidos no concuerdan con la complejidad calculada. Esto puede deberse a que los tiempos se han medido sobre la clase Devorador, que además de ejecutar el algoritmo realiza otras tareas como leer los archivos .json y rellenar los Map con los datos, y para valores grandes no se adapta a la complejidad del algoritmo que calcula la solución al problema.

# Práctica 5

En la clase Ferry hay 3 estructuras de datos:

- Vehicles: guarda los vehículos que se intentarán meter en el barco
- Sumatorio: guarda la suma de las longitudes de los vehículos en orden.
- dp: guarda las posibles soluciones del algoritmo

dp es un array bidimensional, siendo las filas el número de coches a meter y las columnas el espacio ocupado en babor para una solución dada.

No sería necesario almacenar la longitud de estribor explícitamente pues se puede calcular como sumatorio[i] – p siendo sumatorio[i] la suma de las longitudes de los i primeros vehículos y p la longitud ocupada de babor.

Para calcular el estado actual a partir de los anteriores se usan 2 formulas.

- Si $p + v_{i-1} <= L$ -> $dp[i][p + v_{i-1}] = True$
- Si sumatorio[i] – p <= L -> dp[i][p] = True

Siendo $v_{i-1}$ la longitud del vehículo que se está en la posición i – 1. Para realizar estos cálculos, anteriormente se pasó por un condicional para comprobar que la posición dp[i - 1][p] de la tabla de soluciones contenía una solución, y asi poder hacer esos cálculos a partir de esta.

El algoritmo tiene una complejidad O(N*L), siendo N el número de vehículos y L la longitud del ferry, pues la tabla a recorrer para ir rellenándola es de dimensión N*L.

```
C:\Users\cient\Desktop\alg_MenendezCeronPedroUO307004\p5>java Ferry test00.txt

Han llegado un total de 5 vehículos (5 viajarán).

Table with calculations:
 V/L   0   1   2   3   4   5   6   7   8   9  10
   0   T   F   F   F   F   F   F   F   F   F   F
   1   T   F   F   F   T   F   F   F   F   F   F
   2   T   F   F   T   T   F   F   T   F   F   F
   3   F   F   F   T   T   T   F   T   T   T   F
   4   F   F   F   F   T   T   T   T   T   T   T
   5   F   F   F   F   F   F   T   T   T   T   T

Possible assignation:
Vehicle 1 (length 4) -- From (0, -3) -- To (1, 4) -- Position: babor -- Port lengh: 4 -- Starboard length: 0
Vehicle 2 (length 3) -- From (1, 4) -- To (2, 4) -- Position: estribor -- Port lengh: 4 -- Starboard length: 3
Vehicle 3 (length 5) -- From (2, 4) -- To (3, 4) -- Position: estribor -- Port lengh: 4 -- Starboard length: 8
Vehicle 4 (length 2) -- From (3, -1) -- To (4, 6) -- Position: babor -- Port lengh: 6 -- Starboard length: 8
Vehicle 5 (length 2) -- From (4, 6) -- To (5, 6) -- Position: estribor -- Port lengh: 6 -- Starboard length: 10
```

# Práctica 6

Tabla de tiempos para el algoritmo con los diferentes ficheros de prueba. Se ha realizado la ejecución sin el optimizador.

| **Test** | **Tiempo (ms)** |
|---|---|
| Test 0 | 477 |
| Test 1 | 3 |
| Test 2 | 3 |
| Test 3 | 2 |
| Test 4 | 5 |
| Test 5 | 6 |
| Test 6 | 2 |
| Test 7 | 3 |
| Test 8 | 3122 |
| Test 9 | FdT |

El algoritmo tiene una complejidad O(n!), pero esta no se puede apreciar en los tiempos obtenidos por varios motivos. Algunos de estos pueden ser que los ficheros de prueba no recojan casos variados a nivel de número de , y por eso salen tiempos tan reducidos y similares en la mayoría de test, o que esos casos se resuelvan con muy pocas llamadas recursivas aplicando podas concretas, como parece ser el caso tras imprimir el número de llamadas (todas menores de 60, menos en el test 8, que alcanza las 650000 aproximadamente).

# Práctica 7

El heurístico propuesto consiste en dividir toda la lista de elementos a insertar en los contenedores por parejas, sumar ambos elementos y dividirlos entre la capacidad. Los resultados obtenidos con estas operaciones se irán sumando para obtener una mejor solución teórica inicial, añadiéndole 1 al resultado de la pareja (habiéndolo truncado primero) si la operación de la pareja quedaba con decimales.

Después, dentro del propio backtracking se va obteniendo un minimo teórico de contenedores adicionales necesarios haciendo la suma *espacioRestante + capacidad − 1* y dividiéndolo entre la capacidad. Si este valor es mayor o igual que el mejor caso que se había encontrado hasta el momento se sale de ese caso.

Los tiempos siguientes han sido obtenidos sin el optimizador

| **Test** | **Tiempo Almacenaje contenedores (ms)** | **Tiempo Almacenaje contenedoresRyP (ms)** |
|---|---|---|
| Test 0 | 477 | 499 |
| Test 1 | 3 | 2 |
| Test 2 | 3 | 2 |
| Test 3 | 2 | 1 |
| Test 4 | 5 | 2 |
| Test 5 | 6 | 2 |
| Test 6 | 2 | 1 |
| Test 7 | 3 | 1 |
| Test 8 | 3122 | 2 |
| Test 9 | FdT | 3 |

Con el heurístico implementado se han mejorado en gran medida los resultados obtenidos en el algoritmo. Ya no solo se reducen levemente los tiempos de los test intermedios, si no que además el nuevo heurístico nos permite ejecutar el test 8 rápidamente y el test 9 pasa de no ejecutarse a alcanzar resultados similares a los de casos con menos elementos.